



中山大學
SUN YAT-SEN UNIVERSITY

本科生课程报告

软件工程化说明文档

寻迹校园失物招领平台

课程名称： 软件工程

项目名称： 寻迹校园失物招领平台

团队名称： 寻迹项目组

项目负责人： 朱梓涵

团队成员： 朱梓涵、周星辰、林洪宽、郭焜华、程文韬

负责人学号： 24325356

文档版本： V1.0

基线日期： 2026 年 7 月 13 日

中山大学

2026 年 7 月

目录

1 文档目的与范围	1
2 迭代过程与任务协同	1
3 仓库组织与责任边界	2
4 文档与接口契约协作	3
5 Git 协作规范	4
6 编码规范与自动检查	5
7 依赖与构建自动化	6
8 测试自动化	6
9 持续集成流水线	7
10 数据库与环境自动化	8
11 典型变更的工程化流程	9
12 结论	9

1 文档目的与范围

本文说明「寻迹」校园失物招领平台在开发过程中采用的自动化与协作化手段，以及这些手段如何作用于需求变更、编码、测试、集成和交付。软件工程化的目的不是增加额外流程，而是把团队共同遵守的规则固化到仓库、工具和流水线中，使不同成员能够按一致方式开发，并让重复检查由工具执行。

表 1: 软件工程化手段总览

方向	采用的手段	在项目中的作用
过程组织	轻量迭代、需求优先级、任务关闭准则	将大型目标拆成可评审、可联调的增量
团队协作	Git 分支、Conventional Commit、Pull Request	隔离并行修改，保留清晰的变更语义
设计协同	需求、API、枚举、数据库文档同步更新	保持前端、后端、AI 服务对同一契约的理解一致
自动检查	Ruff、Mypy、vue-tsc、Pytest、Vitest	自动发现格式、类型和业务逻辑问题
持续集成	GitHub Actions 五类并行任务	在 push 和 Pull Request 时统一执行质量检查
可重复构建	uv/pnpm 锁文件、Docker、Compose	固定依赖解析方式并统一运行环境
数据演进	SQLAlchemy 模型与 Alembic 迁移	让数据库结构随代码按版本演进

2 迭代过程与任务协同

2.1 轻量迭代方式

项目采用短周期增量开发。需求按 P0、P1、P2 排序：P0 对应登录、发布、搜索、匹配、认领、交接和审核等主流程；P1 处理信用、举报、统计和内容治理；P2 用于不影响主流程的体验增强。每轮迭代从可演示的用户场景出发，同时确定接口、数据、页面和测试任务，避免不同模块各自推进后再集中拼接。

表 2: 迭代阶段与协作活动

阶段	主要目标	团队活动	自动化配合
需求整理	明确角色、场景和优先级	统一术语、状态和验收条件	文档格式与链接检查
设计协同	确定模块、接口和数据关系	评审 API、枚举、状态机和迁移方案	类型检查与迁移逻辑测试
功能开发	实现可联调的纵向功能	分支开发、小步提交、交叉 review	单元测试、静态检查
集成联调	串联双前端、主后端和 AI 服务	按用户场景联调并登记问题	前端构建、Compose 配置检查
交付整理	固化版本、说明和演示环境	更新文档、脚本与部署配置	CI 全量检查、PDF 自动构建

2.2 任务关闭准则

一个开发任务进入合并流程前，需要同时满足以下准则：实现与需求编号或问题描述一致；接口、枚举、数据库字段和工作流变化同步修改文档；代码通过对应模块的格式、类型和测试命令；数据库变化附带 Alembic 迁移；界面变化说明交互影响；Pull Request 聚焦一个主题并由其他成员复核。该准则把「写完代码」转换为团队可共同判断的结束条件。

团队按周协调任务。周初确定本轮用户场景和接口依赖，周中同步跨模块变化及联调问题，周末按场景走查增量。问题记录包含触发条件、预期行为和实际表现，使负责成员可以直接定位，而不依赖口头转述。

3 仓库组织与责任边界

项目采用 monorepo 管理用户端、管理端、主后端、AI 服务、部署文件和文档。相关变更可以在同一个 Pull Request 中评审，例如新增一个状态时，可同时查看后端 Schema、前端共享类型、数据库迁移和接口说明，减少跨仓库版本不同步。

表 3: 模块责任与协作边界

模块	主要责任	协作约束
用户端	用户登录、物品发布、搜索、认领与交接交互	通过统一 API 层访问后端，共用接口类型与枚举
管理端	内容审核、举报治理、用户管理和统计	管理员权限由后端判定，页面不复制业务规则
主后端	HTTP 接口、业务状态、事务与数据持久化	遵循 router、service、repository 分层
AI 服务	分类、匹配和敏感内容辅助判断	通过受鉴权 HTTP 接口调用，不直接访问业务数据库
部署模块	环境模板、镜像构建和六服务编排	配置从环境注入，不把本地秘密写入镜像
文档模块	需求、设计、API、测试和报告	契约变化与实现置于同一变更中评审

主后端的分层规则是工程约束的重要部分。Router 负责 HTTP 参数、Schema 和身份依赖；Service 负责业务规则、状态迁移和事务边界；Repository 负责 SQL 查询与持久化。ORM 模型和 Pydantic Schema 分开维护，使数据库结构、传输格式和业务逻辑各自具有明确职责。

4 文档与接口契约协作

4.1 统一约定

团队把跨模块信息视为共同契约。JSON 字段使用 camelCase，数据库列使用 snake_case，标识符使用 ULID；响应包、分页、时间格式和错误码采用统一约定。角色、物品状态、认领状态和审核状态使用权威枚举，前端展示值不能自行创造新的字面量。

表 4: 契约变更的同步范围

变更类型	需要同步的对象	复核重点
API 请求或响应	API 说明、Pydantic Schema、前端 TypeScript 类型	字段命名、必填性和错误语义
枚举或状态机	枚举说明、Service 守卫、页面标签与操作入口	合法迁移和角色权限
数据库结构	ORM、Alembic 迁移、数据库设计说明	默认值、索引、唯一约束与历史数据处理
业务规则	需求条目、Service、测试用例和交互说明	正常分支、拒绝条件和副作用
页面流程	页面清单、路由、API 调用和验收场景	前后端状态是否一致

4.2 变更影响链

对跨端需求，团队按「需求场景 → 页面操作 → API Schema → Service 状态迁移 → 数据约束 → 自动化测试」的顺序分析影响。例如调整交接确认规则时，需要同时检查认领详情页、确认接口、认领状态机、积分副作用和通知逻辑，而不是只修改一个按钮或一个接口。

5 Git 协作规范

5.1 分支与提交

`main` 用于稳定交付，`develop` 用于团队集成，功能开发和问题修复分别使用 `feature/<name>` 与 `fix/<name>`。成员在独立分支中小步提交，减少多人直接修改同一区域造成的冲突。提交信息采用 Conventional Commit，使历史能够按功能、修复、测试、文档和工程配置快速筛选。

```
feat(FR-02): add lost item publish api
fix(FR-05): correct claim review transition
test(match): cover fallback calculation
docs: update database model
chore(ci): adjust automated checks
```

5.2 Pull Request 协作

Pull Request 关联需求或问题，说明变更范围、受影响模块和本地检查方式。API、枚举、数据库或 workflow 变化必须同时包含对应说明；界面变化附关键页面截图。评审者重点检查业务边界、权限、事务、错误处理和契约一致性。提出修改后，作者以新增提交回应，保留讨论上下文。

仓库通过 `.gitattributes` 统一文本行为：一般文本使用 LF，Windows 命令脚本保持 CRLF，图片、PDF、字体和压缩包标记为二进制。`.gitignore` 排除环境秘密、依赖目录、缓存、覆盖率和构建中间产物，使评审内容集中在源代码与必要交付物上。

6 编码规范与自动检查

6.1 Python 模块

表 5: Python 工程规则与工具

主题	项目约定	自动化手段
格式与风格	Python 3.12、100 列、统一 import 与命名	Ruff lint 与 format check
类型	公共函数和返回值使用类型注解，Schema 明确字段约束	Mypy strict、Pydantic plugin
异步 I/O	数据库和 HTTP 调用使用显式异步接口	Ruff ASYNC、Mypy、异步测试
分层依赖	Router 不直接访问 Repository，模型与 DTO 分离	包结构、代码评审与模块测试
数据一致性	多表写入由 Service 管理事务，关键并发使用锁与唯一约束	Service 测试、数据库迁移

6.2 前端模块

双前端使用 Vue 3 Composition API、TypeScript strict、Pinia 和 Vue Router。共享模块保存接口类型、枚举、标签与格式化函数，两个应用引用同一份源码。统一 Axios 封装负责 API 基址、Bearer token、响应解包、业务错误和未授权会话清理；页面组件只处理展示与交互，权限和最终业务判断仍由后端负责。

`vue-tsc --noEmit` 检查模板和 TypeScript 类型，Vitest 执行工具函数、API 封装和关键交互逻辑测试，Vite production build 检查生产构建。三类检查同时进入前端 CI 任务，使开发机与远端流水线采用相同命令。

7 依赖与构建自动化

Python 模块使用 uv 管理 `pyproject.toml` 和 `uv.lock`，两个前端使用 pnpm 管理 `package.json` 和各自的 `pnpm-lock.yaml`。声明文件表达依赖范围，锁文件固定实际解析版本；CI 采用冻结安装，避免自动更新依赖导致不同成员或不同日期的构建结果漂移。

表 6: 模块构建约定

模块	运行环境	依赖工具	构建或启动方式
主后端	Python 3.12	uv + uv.lock	uvicorn 启动，Docker 构建镜像
AI 服务	Python 3.12	uv + uv.lock	独立 uvicorn 进程，Docker 构建镜像
用户端	Node 20、pnpm 9	pnpm lockfile	Vite 构建，Nginx 提供静态文件
管理端	Node 20、pnpm 9	pnpm lockfile	Vite 构建，Nginx 提供静态文件
完整环境	Docker/Podman Compose	镜像 tag 与环境模板	编排六个服务并创建持久卷

8 测试自动化

测试按模块职责分层。主后端使用 Pytest、pytest-asyncio 和 httpx 的 ASGI transport，覆盖 Service 规则、状态变化和接口响应；AI 服务使用 Pytest 与 HTTP mock，检查输入 Schema、规则计算、服务鉴权和外部调用处理；双前端使用 Vitest 检查共享工具、API 层和关键交互。测试所需的数据库会话、对象存储客户端和外部 HTTP 调用由 fixture 或 mock 隔离，使日常测试不依赖手工搭建共享环境。

表 7: 自动化检查的职责分工

检查层次	工具	主要发现的问题
代码风格	Ruff format、Ruff lint	格式差异、错误导入、常见 Python 问题
静态类型	Mypy、vue-tsc	参数与返回类型、模板绑定、共享接口不一致
模块测试	Pytest、Vitest	业务规则、状态迁移、工具函数和错误处理
构建检查	Vite build、Docker build	生产包编译和镜像构建问题
部署检查	Docker Compose config	环境变量展开、服务定义和编排语法问题

9 持续集成流水线

9.1 触发规则与执行结构

GitHub Actions 在 main、develop 的 push 和 Pull Request 上触发。流水线按 Backend、AI Service、User App、Admin Web 和 Docker Build 五类任务并行执行；同一分支有新提交时取消旧运行，避免过时任务继续占用资源。任何任务失败都会使本次检查失败，所有任务成功后才形成可合并的统一判断。

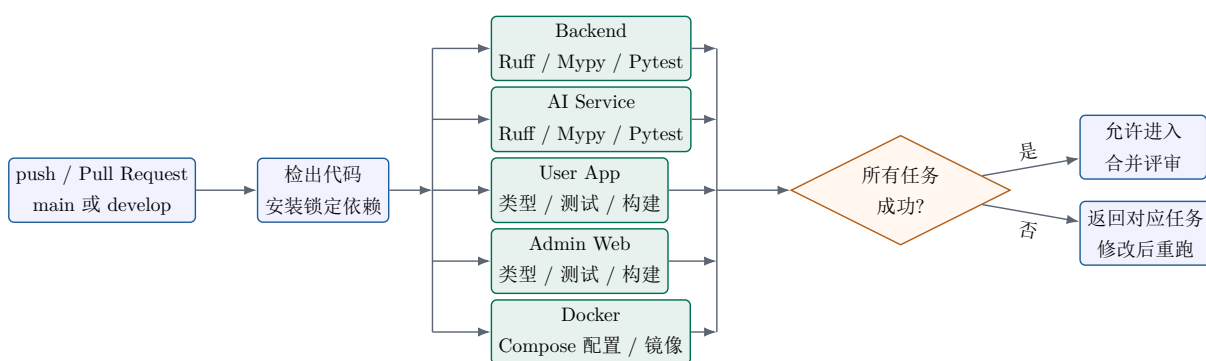


图 1: 持续集成流水线

9.2 各任务采用的自动化步骤

表 8: CI 任务与检查内容

任务	自动化步骤	输出用途
Backend	uv 冻结安装; Ruff lint/format; Mypy; Pytest coverage	统一 Python 规范、类型、业务测试和覆盖率记录
AI Service	uv 安装; Ruff lint/format; Mypy; Pytest	检查独立服务的代码、类型和接口行为
User App	pnpm 冻结安装; vue-tsc; Vitest; Vite build	检查用户端类型、逻辑和生产包
Admin Web	pnpm 冻结安装; vue-tsc; Vitest; Vite build	检查管理端类型、逻辑和生产包
Docker Build	Compose 配置展开; Backend 与 AI 镜像构建	检查部署配置和镜像构建链

Backend 任务生成覆盖率 XML 作为流水线产物，便于查看未覆盖模块；Compose 检查使用专用占位配置验证必填环境变量和编排语法，不使用开发者本地的环境文件。流水线只负责提供一致判断，具体修改仍由对应模块成员在原分支完成。

10 数据库与环境自动化

10.1 数据库迁移

数据库结构通过 SQLAlchemy 模型和 Alembic 迁移共同管理。开发者修改模型后生成迁移初稿，再人工检查字段类型、默认值、索引、唯一约束和历史数据转换。部署时先执行 `alembic upgrade head`，再启动主后端；迁移历史保持单一 head，使不同环境能够从明确版本按顺序升级。

涉及数据结构的 Pull Request 同时包含 ORM、迁移和设计说明。自动生成只负责比较结构，评审者负责判断数据语义。例如建立唯一约束前，需要先处理历史重复记录；修改状态字段时，需要给出旧值到新值的转换规则。

10.2 环境与容器

运行配置由环境变量注入，仓库保留不含真实秘密的示例模板。本地完整栈由 Compose 编排 MySQL、MinIO、AI Service、Backend、User Web 和 Admin Web；后端容器

在启动应用前执行迁移，前端容器使用 Nginx 提供静态资源并代理 API。镜像构建和本地启动采用同一依赖锁文件，减少「开发机可以运行、交付环境不能运行」的差异。

11 典型变更的工程化流程

表 9: 典型变更处理流程

变更类型	处理步骤	自动检查
新增 API	确认需求与错误语义；编写 Schema、Service、Router；同步前端类型和接口说明；补充测试	Ruff、Mypy、Pytest、vue-tsc、Vitest
修改状态机	更新合法迁移；检查权限和副作用；同步枚举与页面操作；覆盖拒绝分支	后端测试、前端类型和构建
修改数据库	更新 ORM；编写 Alembic；说明数据转换；检查单一 head；同步设计文档	迁移检查、Pytest、Docker build
调整页面	确认用户场景；修改组件、Store 与 API 调用；检查加载、空态和错误反馈	vue-tsc、Vitest、Vite build
调整部署	修改环境模板或 Compose；说明变量含义、默认值和服务依赖	Compose config、Docker build

当自动检查失败时，团队按任务名称定位到模块，再根据日志修正实现或测试。修正仍提交到原分支，由流水线重新执行完整任务。这个反馈过程将代码评审和机器检查组合起来：工具处理重复且可判定的规则，成员集中评审需求理解、架构边界和业务语义。

12 结论

本文说明了寻迹项目实际采用的软件工程化方法。自动化方面，项目使用依赖锁、静态检查、类型检查、模块测试、生产构建、容器构建、Compose 配置检查和 Alembic 迁移，并通过 GitHub Actions 统一执行；协作化方面，项目使用轻量迭代、任务关闭准则、模块责任边界、分支与提交规范、Pull Request 评审以及文档和代码同步机制。

这些方法覆盖从需求进入开发到代码合并和环境构建的全过程，使团队能够并行工作，并以一致的规则处理接口、数据、测试和交付变化。